

CMPT 479/982: GPU Architecture & Programming

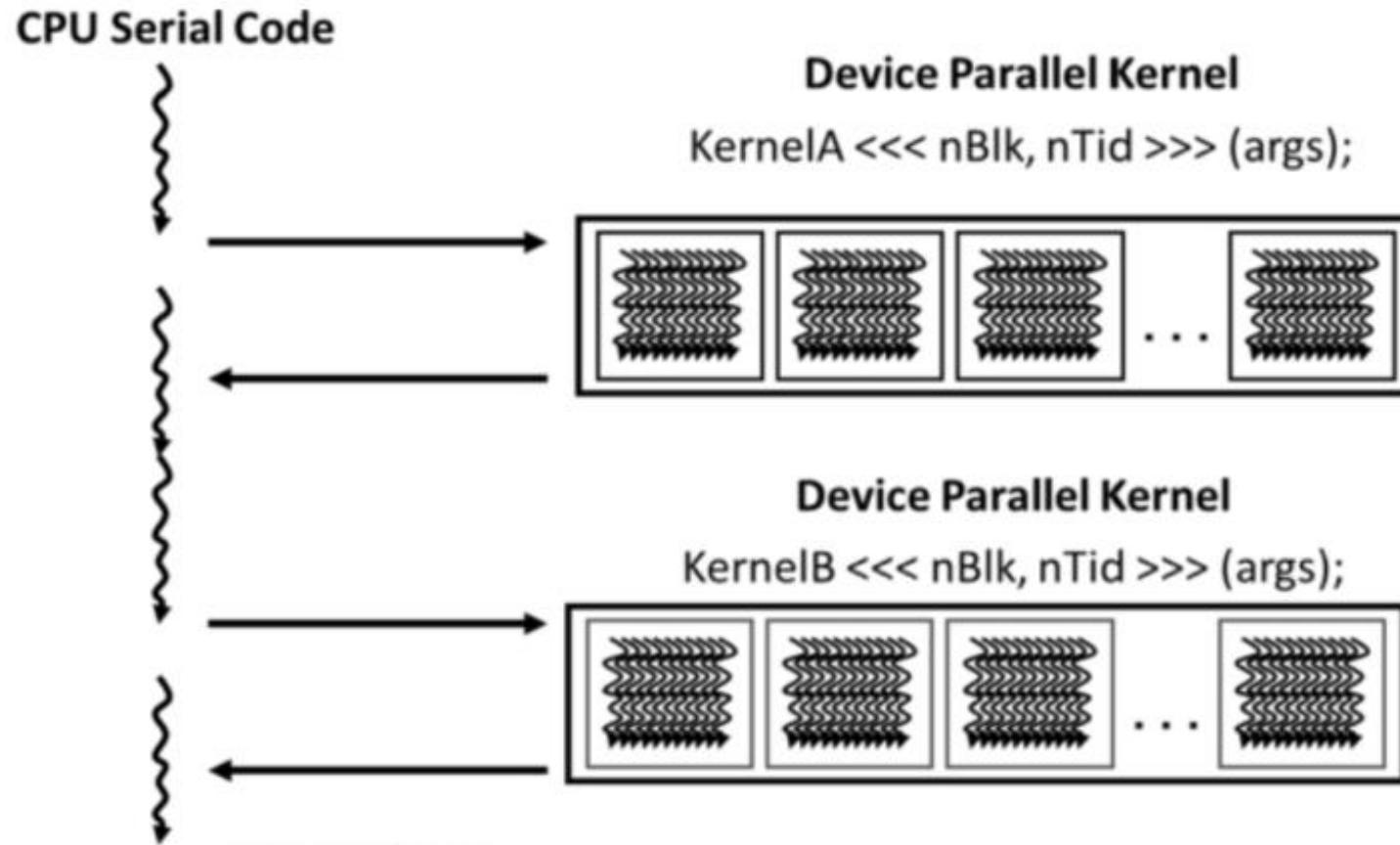
Fall 2026

Multidimensional Grids

GPU Architecture & Thread Scheduling

Alaa Alameldeen (alaa@sfu.ca)

Recap: Execution of a CUDA Program on CPU & GPU



Courtesy: PMPP Textbook

FIGURE 2.3 Execution of a CUDA program.

- Ideally, all sequential code runs on CPU, and all data parallel code runs on GPUs as “device parallel kernels”

Vector Addition Example Recap

Courtesy: PMPP Textbook

```
01 // Compute vector sum C = A + B
02 // Each thread performs one pair-wise addition
03 __global__
04 void vecAddKernel(float* A, float* B, float* C, int n) {
05     int i = threadIdx.x + blockDim.x * blockIdx.x;
06     if (i < n) {
07         C[i] = A[i] + B[i];
08     }
09 }
```

FIGURE 2.10 A simple vector addition kernel function.

```
01 void vecAdd(float* A, float* B, float* C, int n) {
02     float *A_d, *B_d, *C_d;
03     int size = n * sizeof(float);
04
05     cudaMalloc((void **) &A_d, size);
06     cudaMalloc((void **) &B_d, size);
07     cudaMalloc((void **) &C_d, size);
08
09     cudaMemcpy(A_d, A, size, cudaMemcpyHostToDevice);
10     cudaMemcpy(B_d, B, size, cudaMemcpyHostToDevice);
11
12     vecAddKernel<<<ceil(n/256.0), 256>>>>(A_d, B_d, C_d, n);
13
14     cudaMemcpy(C, C_d, size, cudaMemcpyDeviceToHost);
15
16     cudaFree(A_d);
17     cudaFree(B_d);
18     cudaFree(C_d);
19 }
```

FIGURE 2.13 A vector addition kernel call statement.

Multi-Dimensional Grids and Data

Multi-Dimensional Grid Organization

- All threads in a grid execute the same kernel function. Threads rely on their coordinates, i.e., `threadIdx`, to distinguish themselves from others and identify the appropriate portion of the data to process.
- All threads in the same block share the same block index (`blockIdx`).
- Execution configuration parameters in a kernel call statement specify the dimensions of the grid (`gridDim`) and the dimensions of each block (`blockDim`)
- Example: Call to `vecAddKernel` with 32 blocks, 128 threads each:

```
dim3 dimGrid(32, 1, 1);  
dim3 dimBlock(128, 1, 1);  
vecAddKernel<<<dimGrid, dimBlock>>>(...);
```

- `dimGrid` and `dimBlock` can be any legal C++ variable name but need to be of type `dim3` (special case for 1D, `dim3` can be omitted like `vecAdd` example)
- Grid and block dimensions can be computed from other variables, for example if total number of vector elements is unknown in advance.

Limits on Grid and Block Dimensions in CUDA C++

- Allowed values of `gridDim.x` range from 1 to $2^{31}-1$
- Allowed values of `gridDim.y` and `gridDim.z` range from 1 to $2^{16}-1$ (65,535)
- All threads in a block share the same `blockIdx.x`, `blockIdx.y`, and `blockIdx.z` values.
- Among blocks, the `blockIdx.x` value ranges from 0 to `gridDim.x-1`, the `blockIdx.y` value ranges from 0 to `gridDim.y-1`, and the `blockIdx.z` value ranges from 0 to `gridDim.z-1`
- A block is a 3D array of threads. For 2D arrays, `blockDim.z` is set to 1. For 1D arrays, both `blockDim.z` and `blockDim.y` are set to 1
- A block has a maximum of 1024 threads, divided across 3 dimensions (product of three dimensions cannot exceed 1024). For example, `blockDim(16, 32, 2)` is valid while `(32, 32, 2)` is invalid

Example 3D Grid

Courtesy: PMPP Textbook

- Created using

```
dim3 dimGrid(2, 2, 1);  
dim3 dimBlock(4, 2, 2);  
KernelFunction<<<dimGrid, dimBlock>>>(...);
```
- Example: Block (1,0) has `blockIdx.y=1` and `blockIdx.x=0`
- Note that the ordering of the block and thread labels is such that highest dimension comes first (reverse ordering of that used in the code for setting configuration parameters where the lowest dimension comes first)

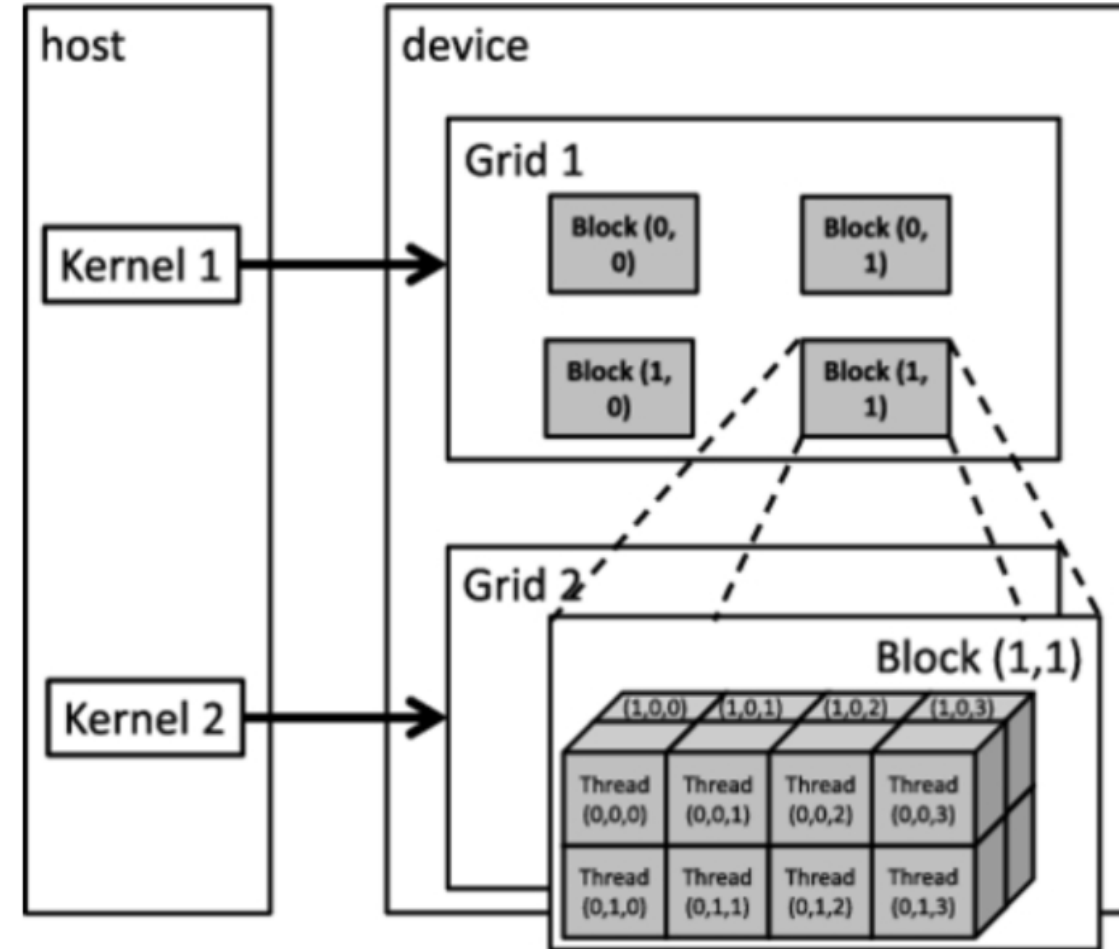


FIGURE 3.1 A multidimensional example of a CUDA grid organization.

Mapping Threads to MultiDimensional Data

- Choice depends on input data
 - Vectors: 1D is a better fit
 - 2D images, matrices: 2D is a better fit
 - 3D images: 3D is a better fit
- Example: Color to Grayscale conversion
 - Input Picture *Pin* is 62x76 pixels: 62 rows in vertical (y) dimension, 76 columns in horizontal (x) direction
 - Assume we use 2D blocks of 16x16 threads. So we need 4 blocks in y direction and 5 blocks in x direction
 - Each thread is assigned to work on a pixel depending on its threadIdx.
 - Vertical (row) coordinate is
$$\text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y}$$
 - Horizontal (column) coordinate is
$$\text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$
 - If needed (not for this problem), the plane (z) coordinate is
$$\text{blockIdx.z} * \text{blockDim.z} + \text{threadIdx.z}$$

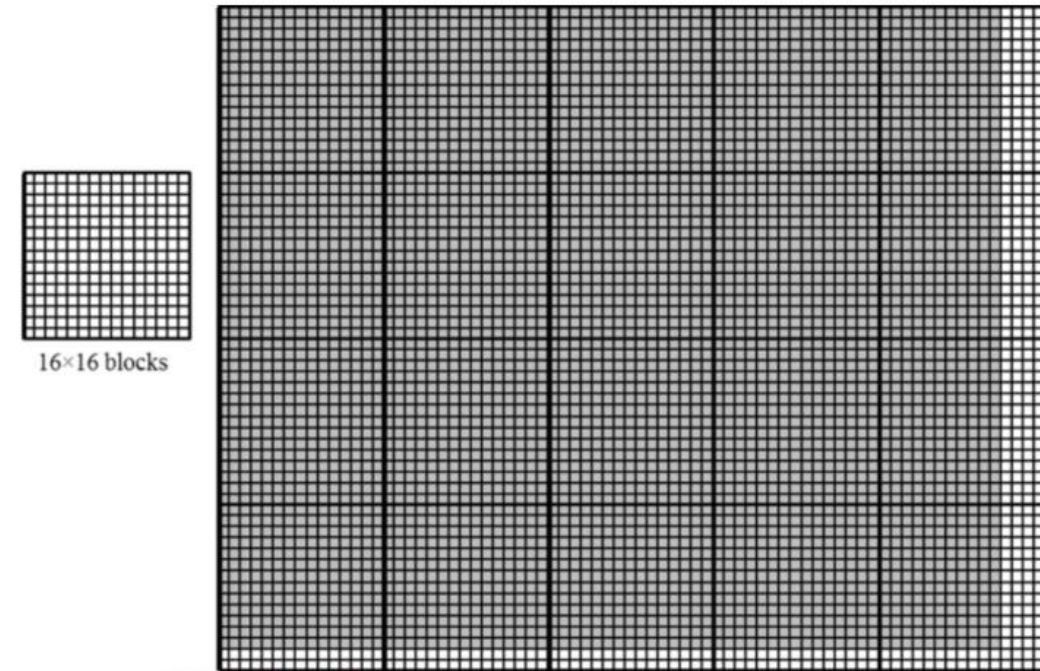


FIGURE 3.2 Using a 2D thread grid to process a 62×76 picture *Pin*.

Courtesy: PMPP Textbook

Mapping Data to Device Memory

Courtesy: PMPP Textbook

- Typically data is mapped to memory in row-major order: All data of first row is stored first, then all data of second row, etc. This results in an equivalent linear (1D) array
 - Column-major order stores all data of first column, then all data of second column etc.

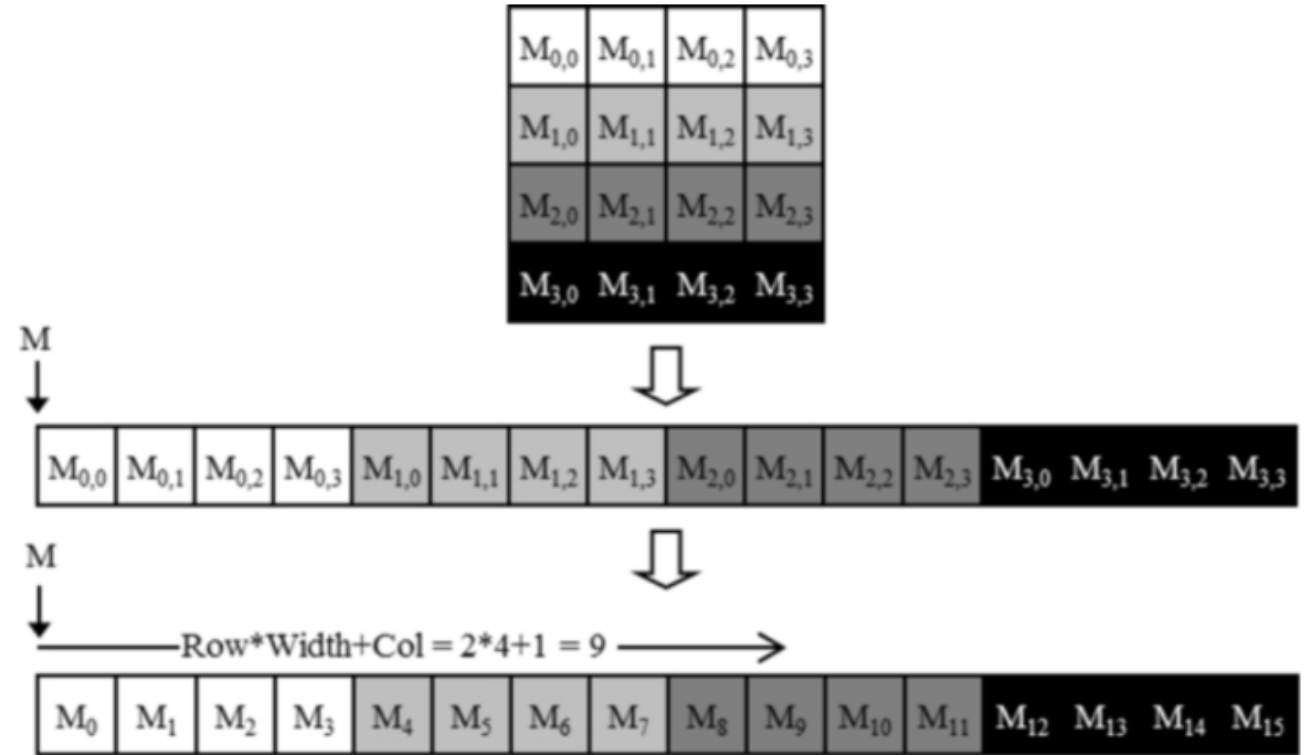


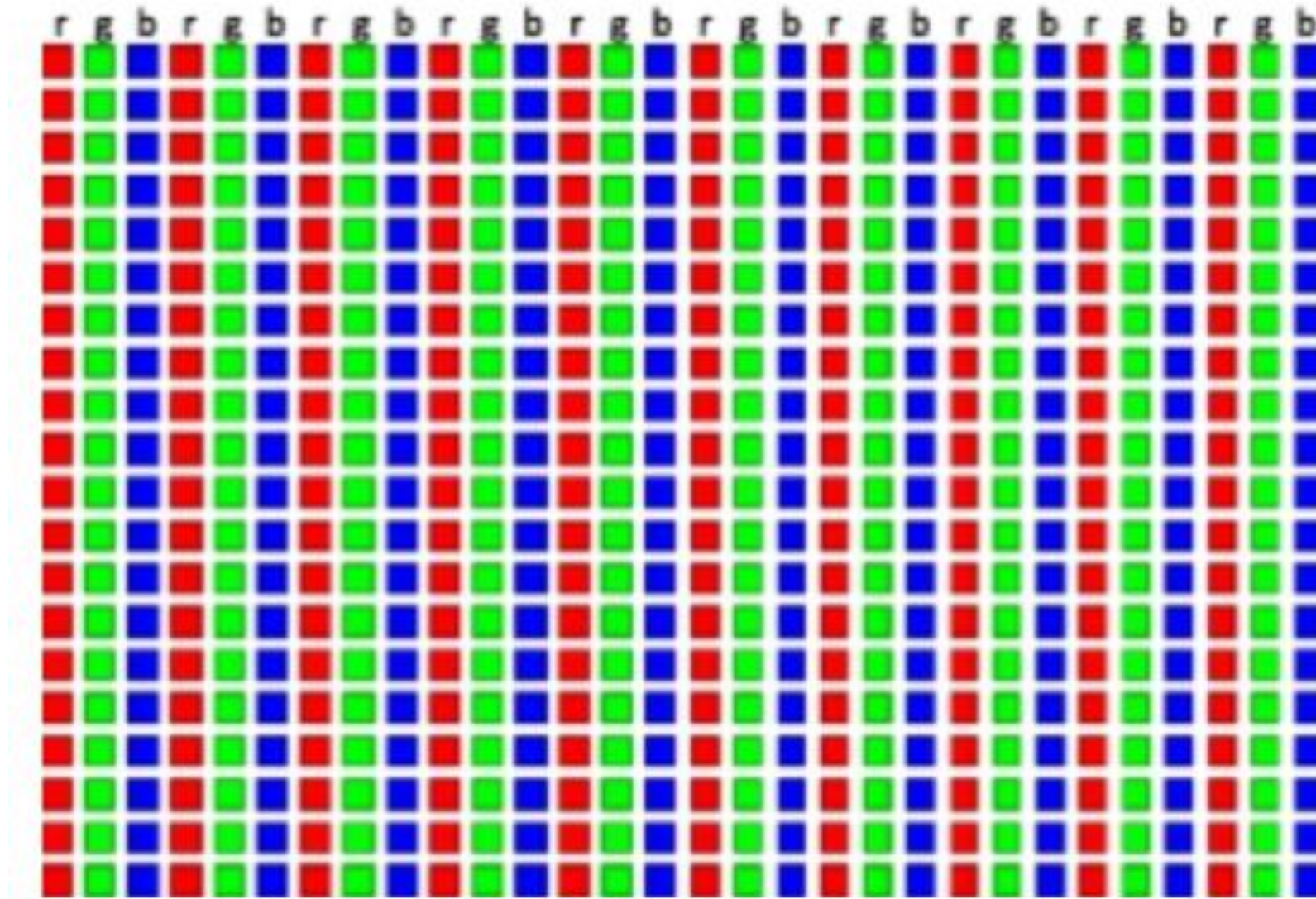
FIGURE 3.3 Row-major layout for a 2D C++ array. The result is an equivalent 1D array accessed by an index expression $j * \text{Width} + i$ for an element that is in the j^{th} row and i^{th} column of an array of *Width* elements in each row.

Color to Grayscale Conversion: Calling Kernel

- 16x16 blocks
- Grid dimensions based on number of rows or columns (n rows, m columns)
- Calling the kernel:

```
dim3 dimGrid(ceil(m/16.0), ceil(n/16.0), 1);  
dim3 dimBlock(16, 16, 1);  
colorToGrayscaleConversion<<<dimGrid, dimBlock>>>(Pin_d, Pout_d, m, n);
```

Input Color is 3 bytes per pixel



Courtesy: PMPP Textbook

Color to Grayscale Conversion Kernel

```
01 // The input image is encoded as unsigned chars [0, 255]
02 // Each pixel is 3 consecutive chars for the 3 channels (RGB)
03 __global__
04 void colorToGrayscaleConversion(unsigned char * Pout,
05                                unsigned char * Pin, int width, int height) {
06     int col = blockIdx.x*blockDim.x + threadIdx.x;
07     int row = blockIdx.y*blockDim.y + threadIdx.y;
08     if (col < width && row < height) {
09         // Get 1D offset for the grayscale image
10         int grayOffset = row*width + col;
11         // One can think of the RGB image having CHANNEL
12         // times more columns than the gray scale image
13         int rgbOffset = grayOffset*CHANNELS;
14         unsigned char r = Pin[rgbOffset]; // Red value
15         unsigned char g = Pin[rgbOffset + 1]; // Green value
16         unsigned char b = Pin[rgbOffset + 2]; // Blue value
17         // Perform the rescaling and store it
18         // We multiply by floating point constants
19         Pout[grayOffset] = (unsigned char) 0.21f*r+0.71f*g+0.07f*b;
20     }
21 }
```

Courtesy: PMPP Textbook

FIGURE 3.4 Source code of
colorToGrayscaleConversion with
2D thread mapping to data.

Boundary Conditions

- Area 1: All threads have an assigned pixel to process
- Area 2: row values are always within range, but col values of some of them exceed 76. So 12 threads in each row will process pixels; while the remaining 4 threads in each row will fail
- Area 3: col values are always within range, but row values for the last two rows exceed 62. So 14 threads in each column will process pixels while the remaining two will fail
- Area 4: 4 threads in each of the top 14 rows will find their col values out of range. The entire bottom two rows will find their row values out of range. Overall, only $14 \times 12 = 168$ of the 256 threads in that will process pixels

Courtesy: PMPP Textbook

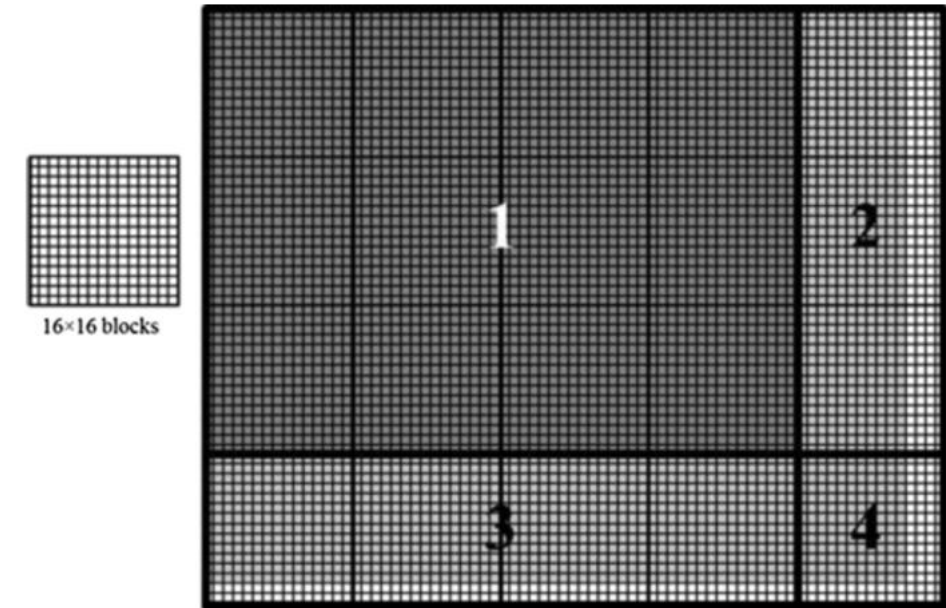


FIGURE 3.5 Covering a 76×62 picture with 16×16 blocks.

Example: Image Blur

Courtesy: PMPP Textbook



FIGURE 3.6 An original image (left) and a blurred version (right).

- Previous examples are simple: Each thread only operates on data that is different from other threads.
- Image Blur is a more complex example where each pixel of output image is a weighted sum of a patch of $N \times N$ pixels including the original pixel in the input image. Simplified version just used arithmetic mean
- Example: 3×3 patch. Original pixel at (row,col) becomes the average of all pixels in the square from (row-1, col-1) to (row+1, col+1)
 - Pixel (40,50) is the average of pixels (39,49), (39, 50), (39, 51), (40, 49), (40, 50), (40, 51), (41, 49), (41, 50), (41, 51)

Image Blur: Threads Mapping

Courtesy: PMPP Textbook

- Similar to “Color to Grayscale” problem, each thread produces one output pixel
- Each thread uses data from original pixel and all pixels around it based on BLUR_SIZE
 - Size of patch is $(2*BLUR_SIZE+1)$.
For example: $BLUR_SIZE = 1$ gets the average of a 3×3 patch around the original pixel

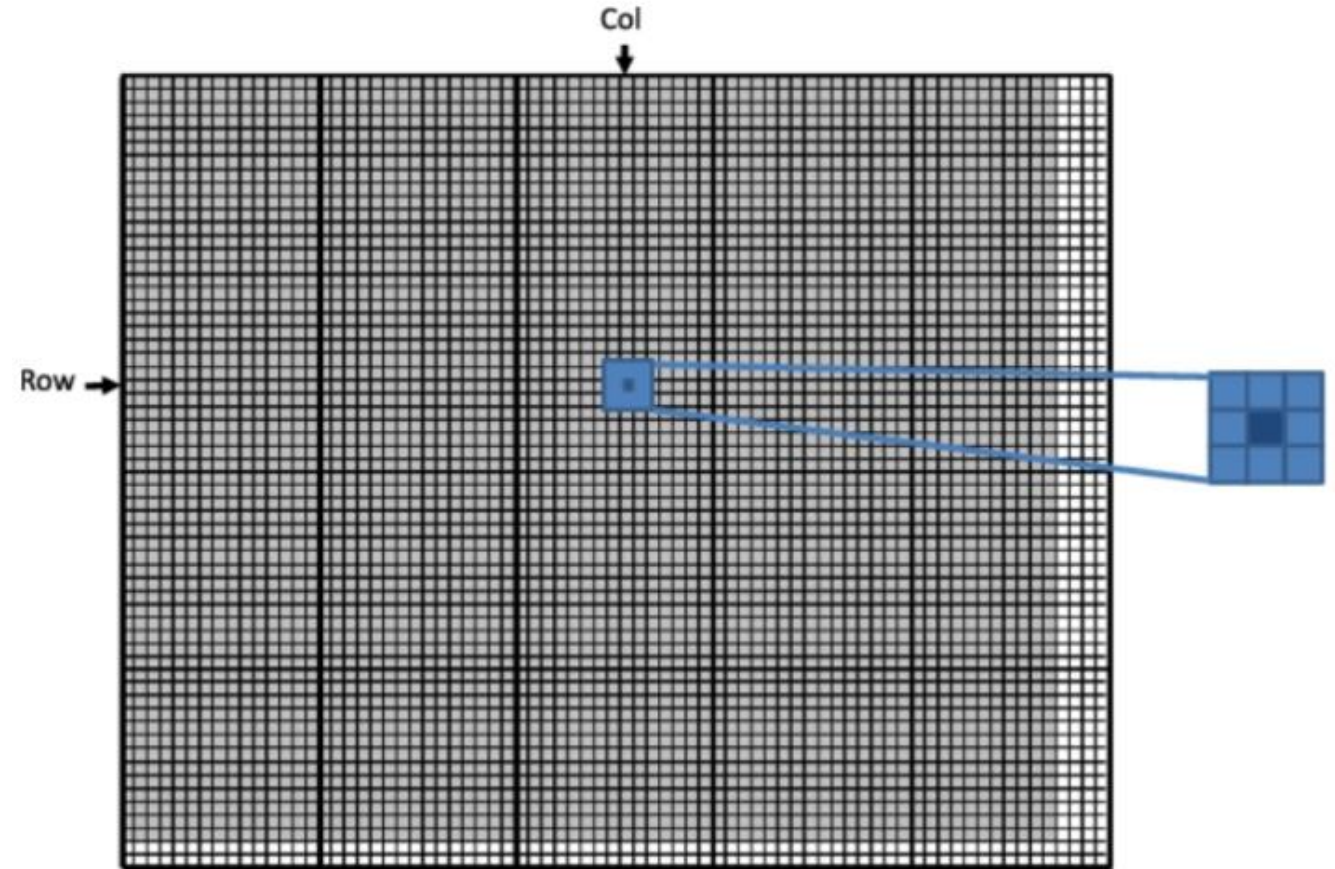


FIGURE 3.7 Each output pixel is the average of a patch of surrounding pixels and itself in the input image.

Image Blur Kernel

Courtesy: PMPP Textbook

Identify pixel from thread info

Why?

```
01  __global__
02  void blurKernel(unsigned char *in, unsigned char *out, int w, int h){
03      int col = blockIdx.x*blockDim.x + threadIdx.x;
04      int row = blockIdx.y*blockDim.y + threadIdx.y;
05      if(col < w && row < h) {
06          int pixVal = 0;
07          int pixels = 0;
08          // Get average of the surrounding BLUR_SIZE x BLUR_SIZE box
09          for(int blurRow=-BLUR_SIZE; blurRow<BLUR_SIZE+1; ++blurRow){
10              for(int blurCol=-BLUR_SIZE; blurCol<BLUR_SIZE+1; ++blurCol){
11                  int curRow = row + blurRow;
12                  int curCol = col + blurCol;
13                  // Verify we have a valid image pixel
14                  if(curRow>=0 && curRow<h && curCol>=0 && curCol<w) {
15                      pixVal += in[curRow*w + curCol];
16                      ++pixels; // Keep track of number of pixels in the avg
17                  }
18              }
19          }
20          // Write our new pixel value out
21          out[row*w + col] = (unsigned char)((float)pixVal/pixels);
22      }
23  }
24 }
```

FIGURE 3.8 An image blur kernel.

Image Blur: Boundary Conditions

Courtesy: PMPP Textbook

- Pixels on the image boundary use an average of a different number of points. Not always $(2*BLUR_SIZE+1)^2$

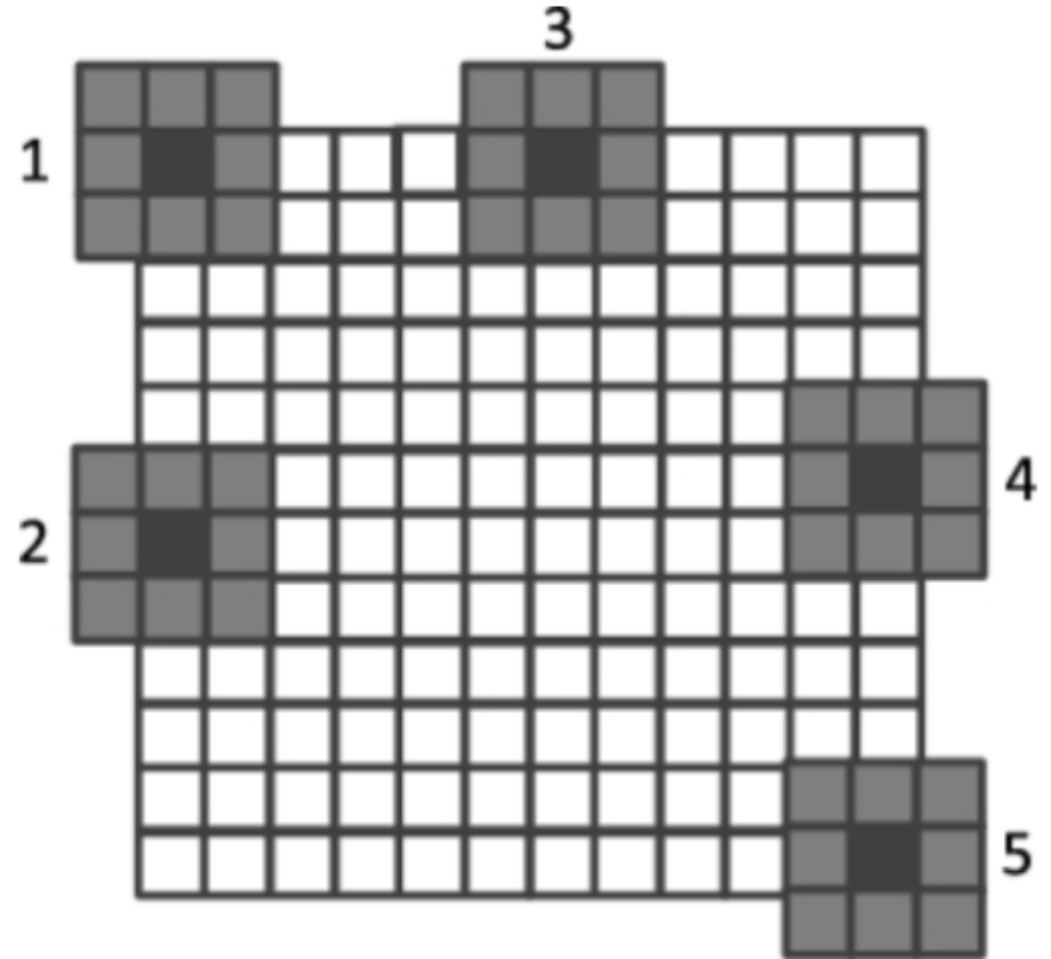


FIGURE 3.9 Handling boundary conditions for pixels near the edges of the image.

Matrix Multiplication: Version 1

Courtesy: PMPP Textbook

- Multiply two square matrices M and N (each dimension = width) and store the output in matrix P
- Each element of the output matrix P is the dot product of one row of M and one column of N
- Simple version: Each thread produces one output element of matrix P. The row and column of the output element is:

```
row = blockIdx.y*blockDim.y  
      + threadIdx.y  
col  = blockIdx.x*blockDim.x  
      + threadIdx.x
```

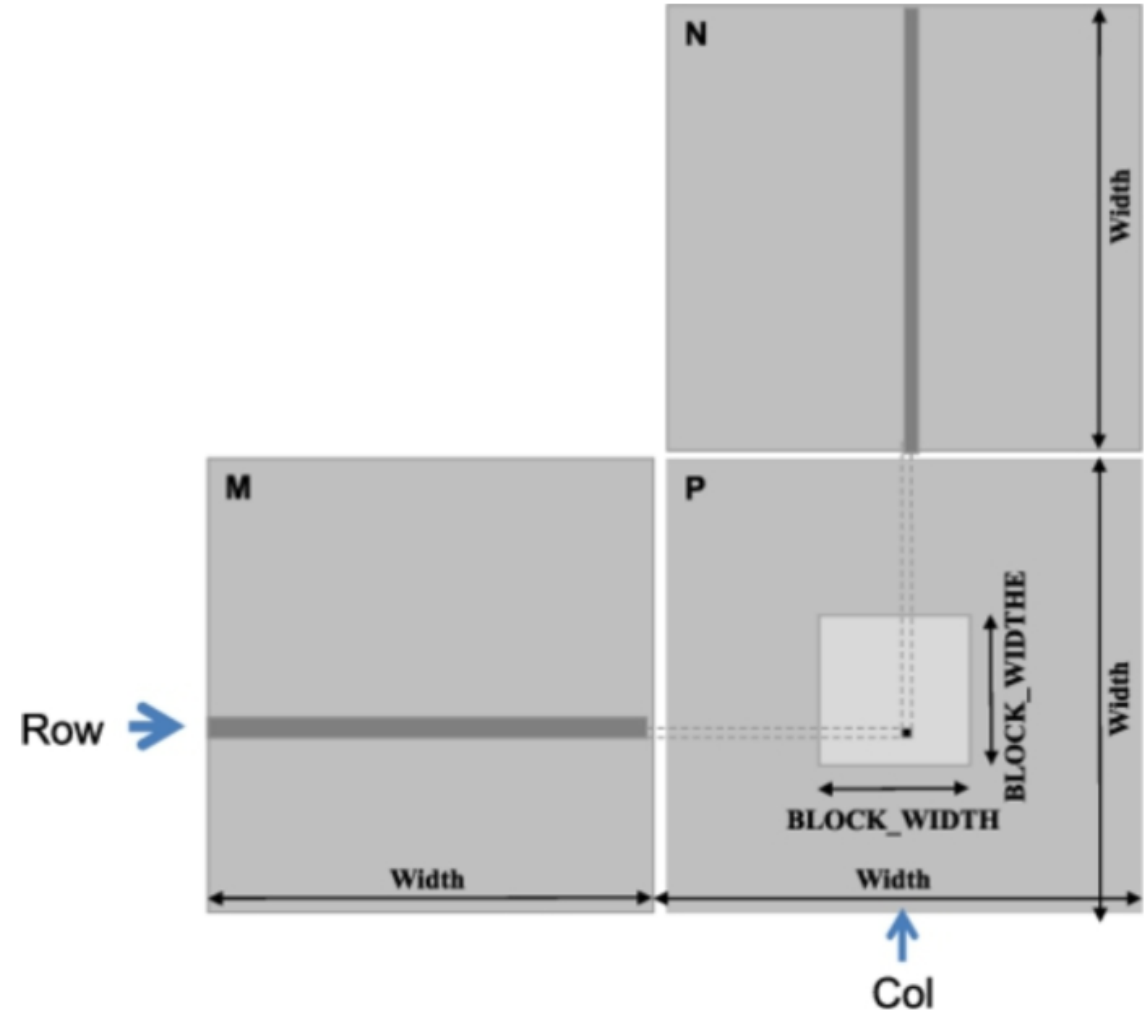


FIGURE 3.10 Matrix multiplication using multiple blocks by tiling P.

Matrix Multiplication Kernel

Courtesy: PMPP Textbook

```
01  __global__ void MatrixMulKernel(float* M, float* N,  
02                                float* P, int width) {  
03      int row = blockIdx.y*blockDim.y+threadIdx.y;  
04      int col = blockIdx.x*blockDim.x+threadIdx.x;  
05      if ((row < width) && (col < width)) {  
06          float Pvalue = 0;  
07          for (int k = 0; k < width; ++k) {  
08              Pvalue += M[row*width+k]*N[k*width+col];  
09          }  
10          P[row*width+col] = Pvalue;  
11      }  
12  }
```

FIGURE 3.11 A matrix multiplication kernel using one thread to compute one P element.

- Host code should be obvious
- Need to check row and column are within bounds
- Accumulate the product of the k^{th} element of each row with the k^{th} element of each column

Demo for a 4x4 Output Matrix

Courtesy: PMPP Textbook

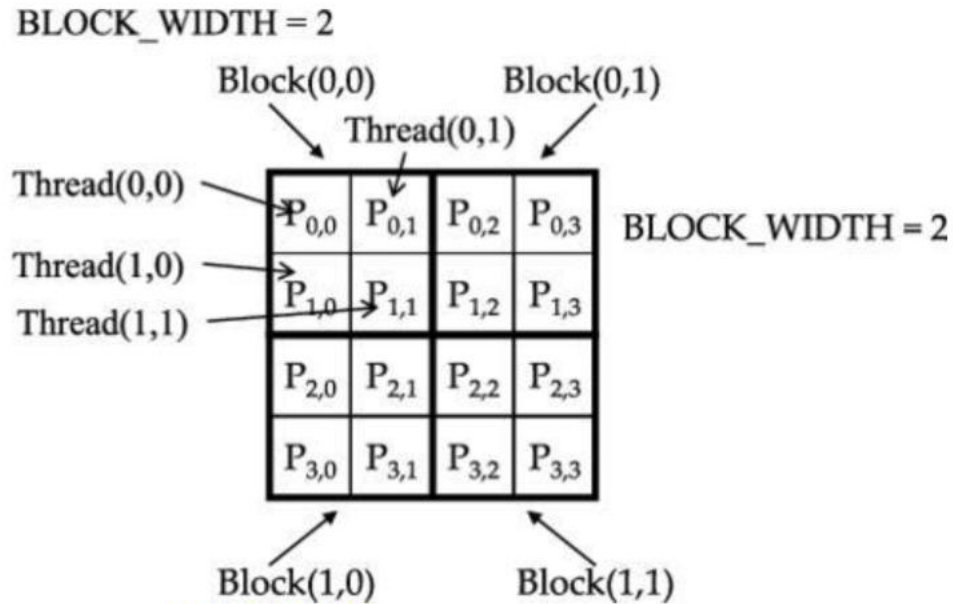


FIGURE 3.12 A small execution example of matrixMulKernel.

- gridDim: 2x2, blockDim: 2x2. Four threads in each block, and 4 blocks in the grid
- Each thread computing one element of the output matrix based on its thread coordinates

```
row = blockIdx.y*blockDim.y + threadIdx.y
```

```
col = blockIdx.x*blockDim.x + threadIdx.x
```

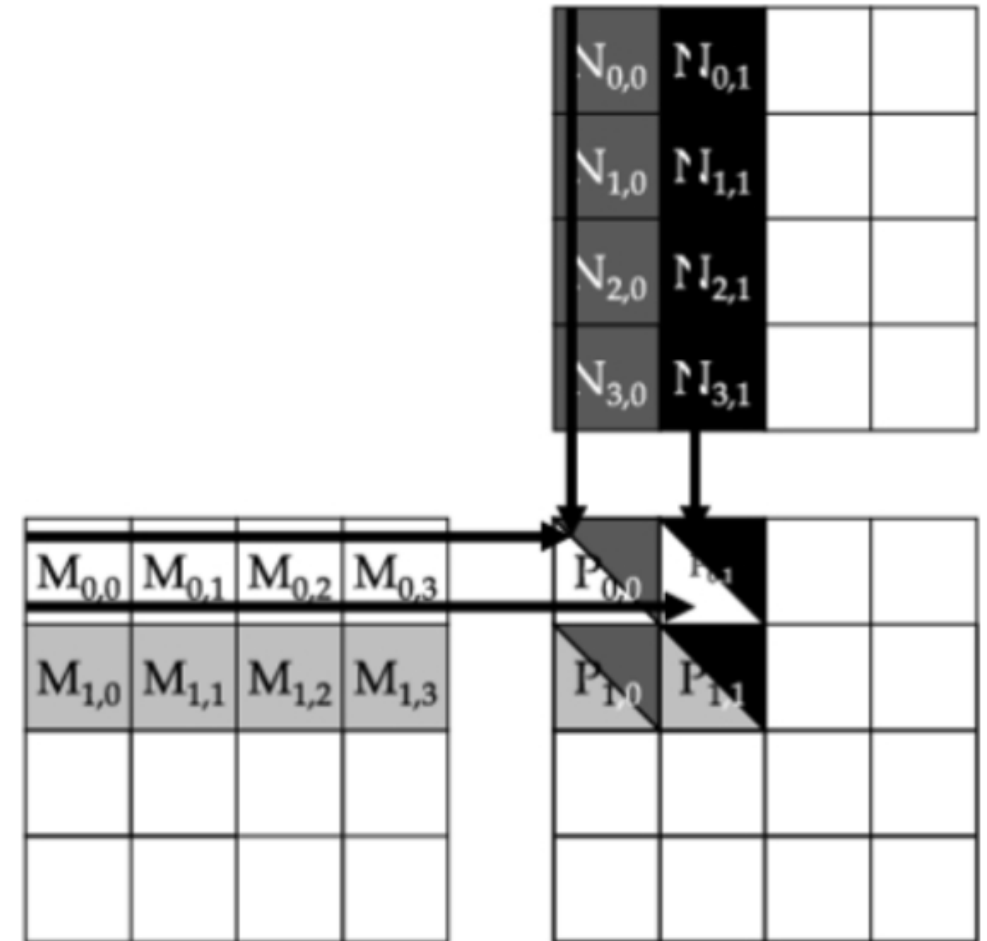


FIGURE 3.13 Matrix multiplication actions of one thread block.

Exercises (from Textbook)

Courtesy: PMPP Textbook

- Consider a two-dimensional matrix with a width of 400 and a height of 500. The matrix is stored as a one-dimensional array. Specify the array index of the matrix element at row 20 and column 10: a. If the matrix is stored in row major order. b. If the matrix is stored in column major order.
- Consider a three-dimensional tensor with a width of 400, a height of 500, and a depth of 300. The tensor is stored as a one-dimensional array in row major order. Specify the array index of the tensor element at $x = 10$, $y = 20$, and $z = 5$?
- Write a kernel that has each thread produce one output matrix row. Fill in the execution configuration parameters for the design.
- Write a kernel that has each thread produce one output matrix row. Fill in the execution configuration parameters for the design.
- Analyze the pros and cons of each kernel design above vs. the code discussed in class

Architecture of a Modern GPU

GPU Compute Architecture

Courtesy: PMPP Textbook

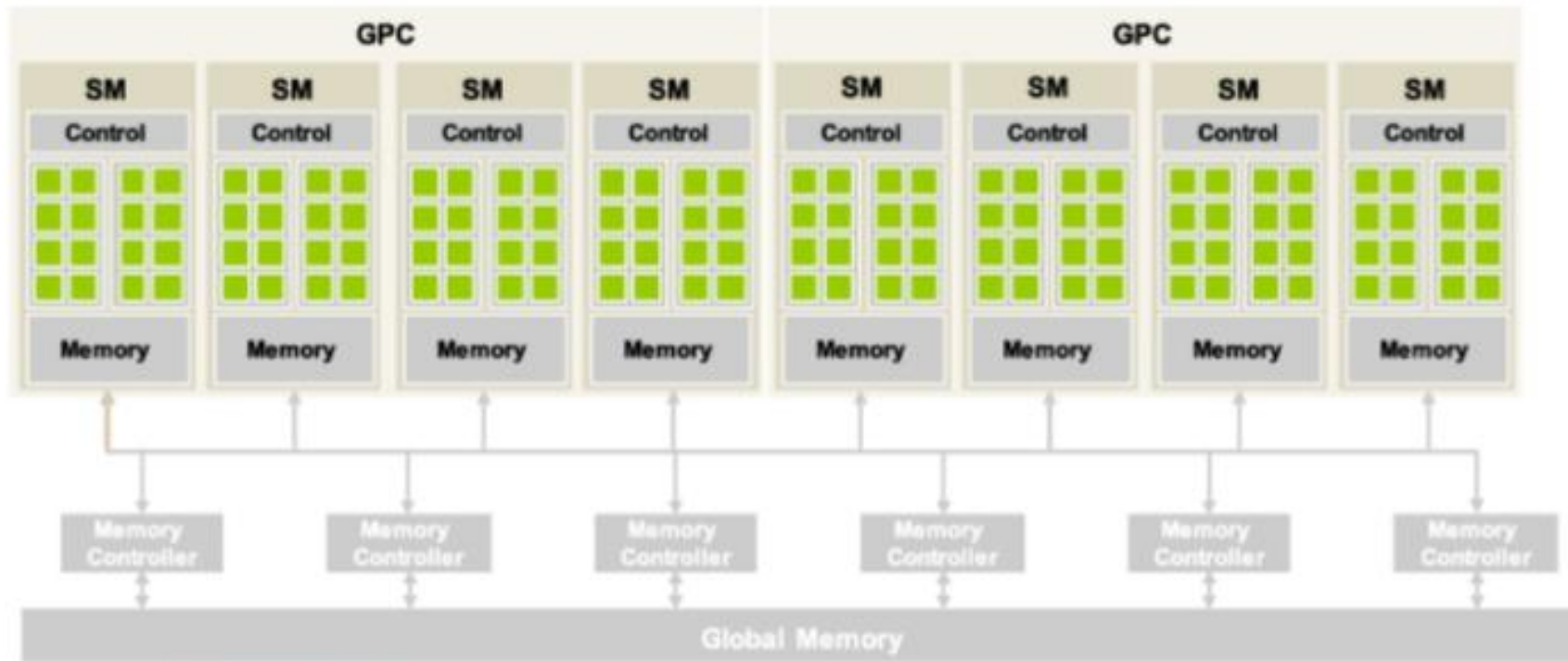


FIGURE 4.1 Architecture of a CUDA-capable GPU.

- GPU includes a few GPU Processing Clusters (GPCs)
- Each GPC has a few Streaming Multiprocessors (SMs). Each SM has a few Streaming Processors (SPs), also known as CUDA cores.
- A few other components include Ray Tracing (RT) cores, Tensor cores, Texture Units, FP64 cores
- On-chip memory resources (next week's topic), global memory with GDDR/HBM memory.

Example: Ada Lovelace Architecture (AD102)



Courtesy: Nvidia Whitepaper “NVIDIA ADA LOVELACE PROFESSIONAL GPU ARCHITECTURE” at https://images.nvidia.com/aem-dam/en-zz/Solutions/technologies/NVIDIA-ADA-GPU-PROVIZ-Architecture-Whitepaper_1.1.pdf

- GPU includes a 12 GPCs each with 12 SMs (144 SMs total), 576 Texture Units organized in Texture Processing Clusters (TPCs)
- Each SM has 128 SP (total 18,432 SPs), 2 RT cores (total 288), 2 FP64 cores (total 288), 4 Tensor Cores (576 total), 256KB register file, 128KB L1/shared memory

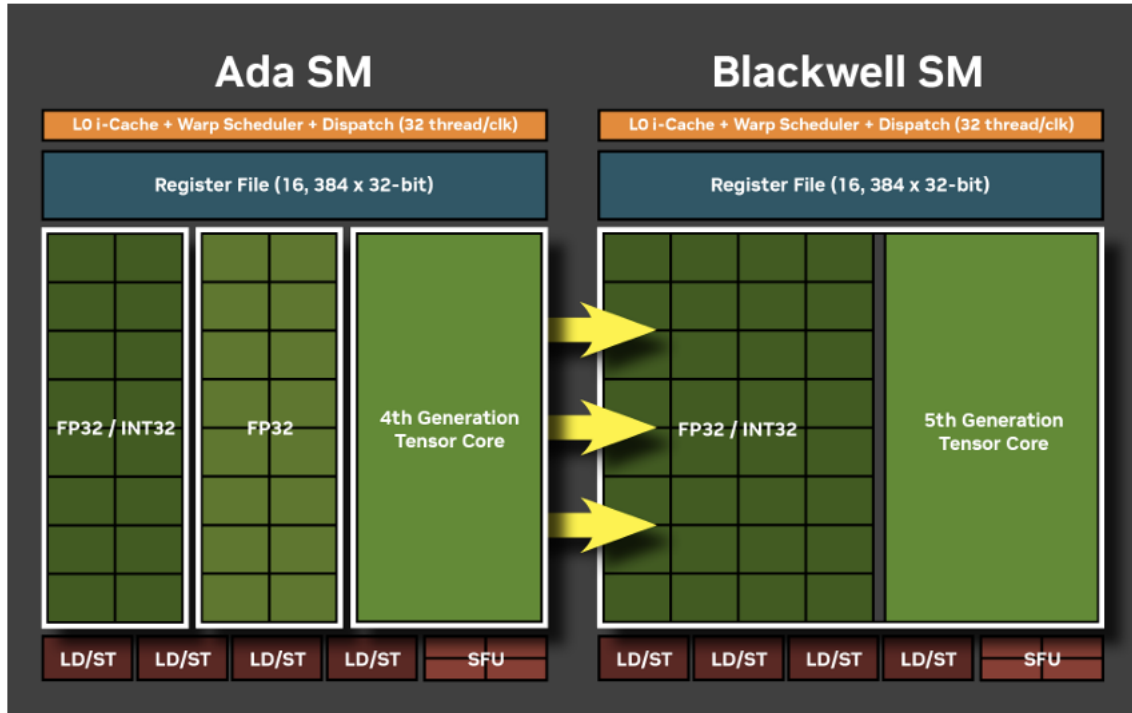
More Recent Blackwell Architecture (GB202)



Courtesy: Nvidia Whitepaper
“NVIDIA RTX BLACKWELL GPU
ARCHITECTURE” at
<https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf>

- GPU includes a 12 GPCs each with 8 Texture Processing Clusters (TPCs, 96 total), each with 2 SMs (16 SMs per GPC, 192 SMs total)
- Each SM has 128 SP (total 24,576 SPs), 1 RT cores (total 192), 2 FP64 cores (total 384), 4 Tensor Cores (768 total), 4 Texture Units (768 total), 256KB register file, 128KB L1/shared memory
- Texture units are responsible for handling texture mapping operations; fetching texels from textures, applying texture filtering, and handling texture coordinates. Texels represent the texture information including colors and patterns that are applied to 3D surfaces, defining the visual appearance of the texture that is applied to the surface of an object.

Blackwell GPC, SMs



- Each SM includes 128 CUDA cores (SPs), each processing FP32/INT32 operations
- LD/ST units
- Special Function Units (SFUs) for math acceleration of operations like square root, exponent, sin/cos and some AI functions, e.g. softmax

Courtesy: Nvidia Whitepaper
“NVIDIA RTX BLACKWELL GPU
ARCHITECTURE” at
<https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf>

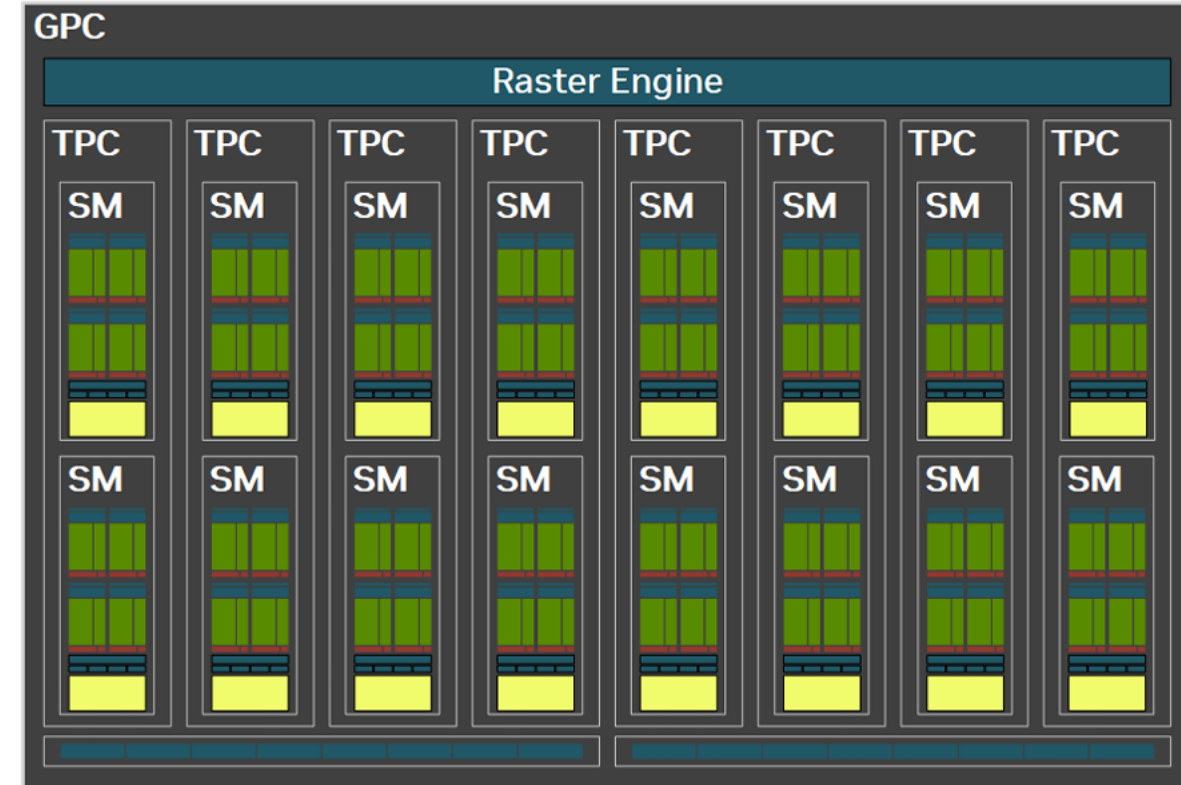


Figure 4. The Blackwell GPC with Raster Engine, 8 TPCs, 16 SMs, and 16 ROPs.

Thread Block Scheduling

Courtesy: PMPP Textbook

- On kernel call, CUDA runtime system launches a grid of threads to execute kernel code
 - Number of threads may require more resources than available on a GPU.
- A thread block scheduling mechanism assigns threads to execution resources when they become available on a block-by-block basis
 - All threads in a block are assigned to the same SM. Multiple blocks could be assigned to the same SM
 - Threads in same block interact with each other differently from those in other blocks (block-wide barriers, fast data exchange through SRAM)
 - optional hierarchy level: thread block cluster, a group of thread blocks co-scheduled on the same GPC, exchange data from distributed shared memory

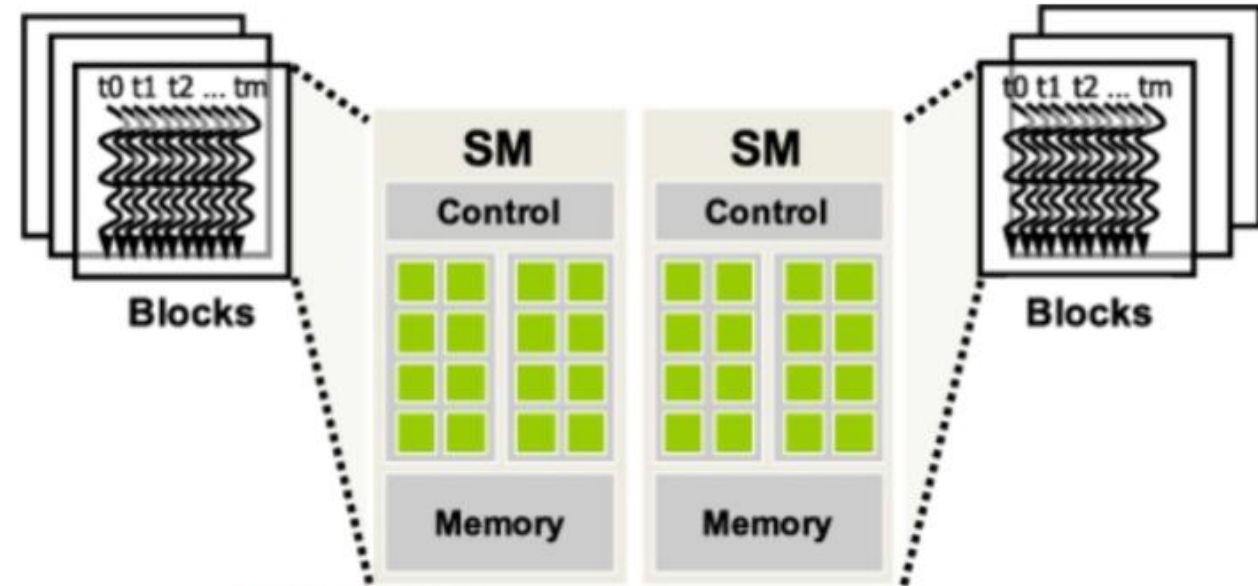


FIGURE 4.2 Thread block assignment to Streaming Multiprocessors (SMs).

Synchronization

- Threads in the same block to coordinate using block-wide barrier synchronization function `__syncthreads()`
 - Starts with two “_” indicating an intrinsic function: A library function from programmer perspective, but compiler translates to special instructions
- Barrier scope indicates collection of threads involved: thread block, thread block cluster, or grid.
- All threads need to arrive at barrier before any of them can make forward progress (released from barrier)
- At the thread block cluster level, Cooperative Groups API provides cluster-wide synchronization. All threads need to arrive at barrier before any can get released from barrier.
- At the grid level, Cooperative Groups API provides grid-wide synchronization, more heavyweight than other types.

Courtesy: PMPP Textbook

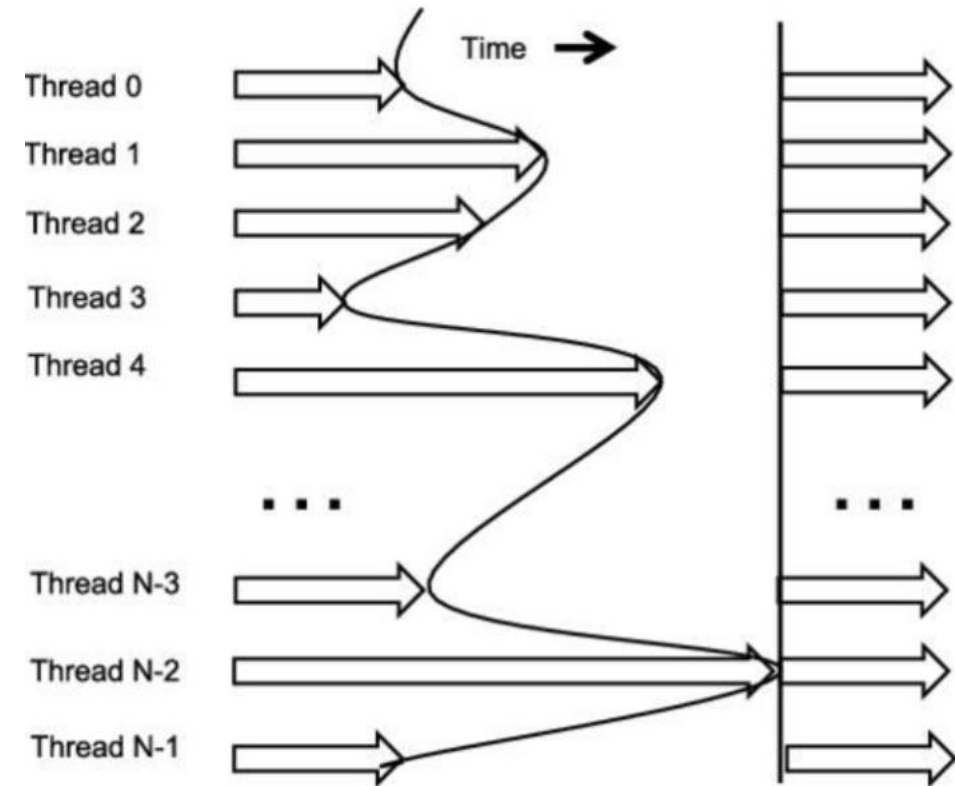


FIGURE 4.3 An example execution timing of barrier synchronization. The arrows represent execution activities over time. The vertical curve marks the time when each thread executes the `__syncthreads` statement. The empty space to the right of the vertical curve depicts the time that each thread waits for all threads to complete. The

Barrier Restrictions

Courtesy: PMPP Textbook

- If `__syncthreads()` is present, it must be executed by all threads in a block
- Incorrect use can cause undefined behavior and possible deadlocks
- Barrier-participating threads should execute in close time proximity to avoid long delays; and the system should ensure all have access to necessary resources to eventually arrive at the barrier:
 - All block threads assigned resources as a unit to the same SM simultaneously
 - A block can only begin execution after all threads in block have resources to execute
- This encourages limited scope for barrier synchronization – no need to synchronize more blocks if only single block is sufficient

```
1 void incorrect_barrier_example(int n) {  
2     ...  
3     if (threadIdx.x % 2 == 0) {  
4         ...  
5         __syncthreads();  
6     }  
7     else {  
8         ...  
9         __syncthreads();  
10    }  
11 }
```

FIGURE 4.4 An incorrect use of `__syncthreads()`.

Why is this wrong?

Barrier Restrictions

- If barriers are within a block, blocks can be executed in any order
- A group of thread blocks executing simultaneously is a **wave**.
- The ratio of the #blocks/grid to number of blocks that can execute simultaneously is **the number of waves**. (use ceiling if not whole number)
- More waves help with load balancing
- Last wave may have fewer blocks. Example: A grid with 660 blocks execute on a GPU that can execute 264 blocks simultaneously: $2.5 \rightarrow 3$ waves with 264, 264, and 132 blocks.
 - Preferable to have an integer #waves
- **Transparent scalability:** Ability to execute the same application code on different hardware with different amounts of execution resources.

Courtesy: PMPP Textbook

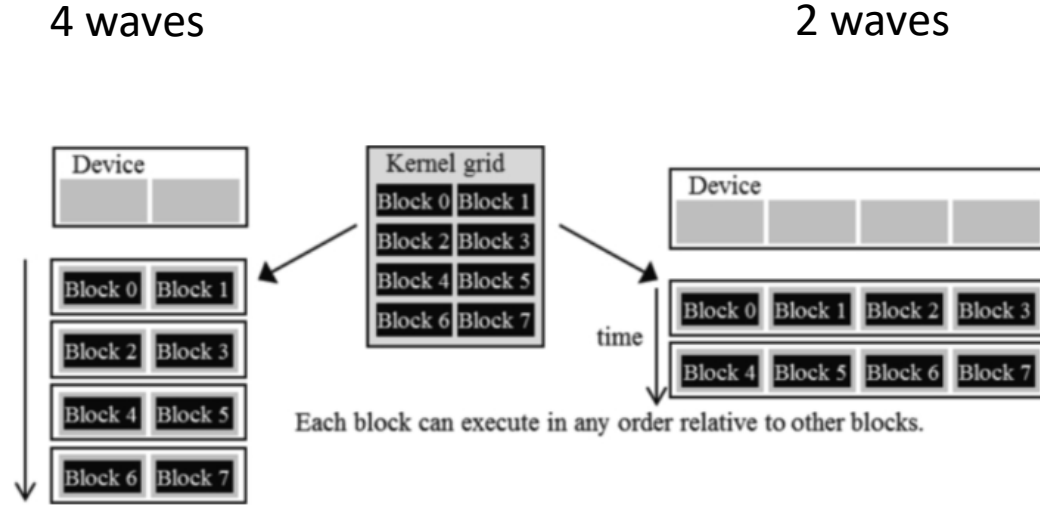


FIGURE 4.5 Lack of synchronization constraints between blocks enables transparent scalability for CUDA programs.

Warps & Thread Scheduling

Courtesy: PMPP Textbook

How are threads within a block scheduled?

- After a block is assigned to an SM, it is further divided into 32-thread units called warps.
- A **warp** is the unit of thread scheduling in SMs.
- Example: 3 blocks assigned to the same SM; each block is divided into 32-thread warps that are scheduled together.
- Each warp has threads with consecutive `threadIdx` values. Warp 0: Threads 0-31; Warp 1: Threads 32-63, etc.
- Assume in this example each block has 256 threads, then each block has $256/32=8$ warps. The SM has 3 blocks each with 8 warps, total 24 warps per SM.

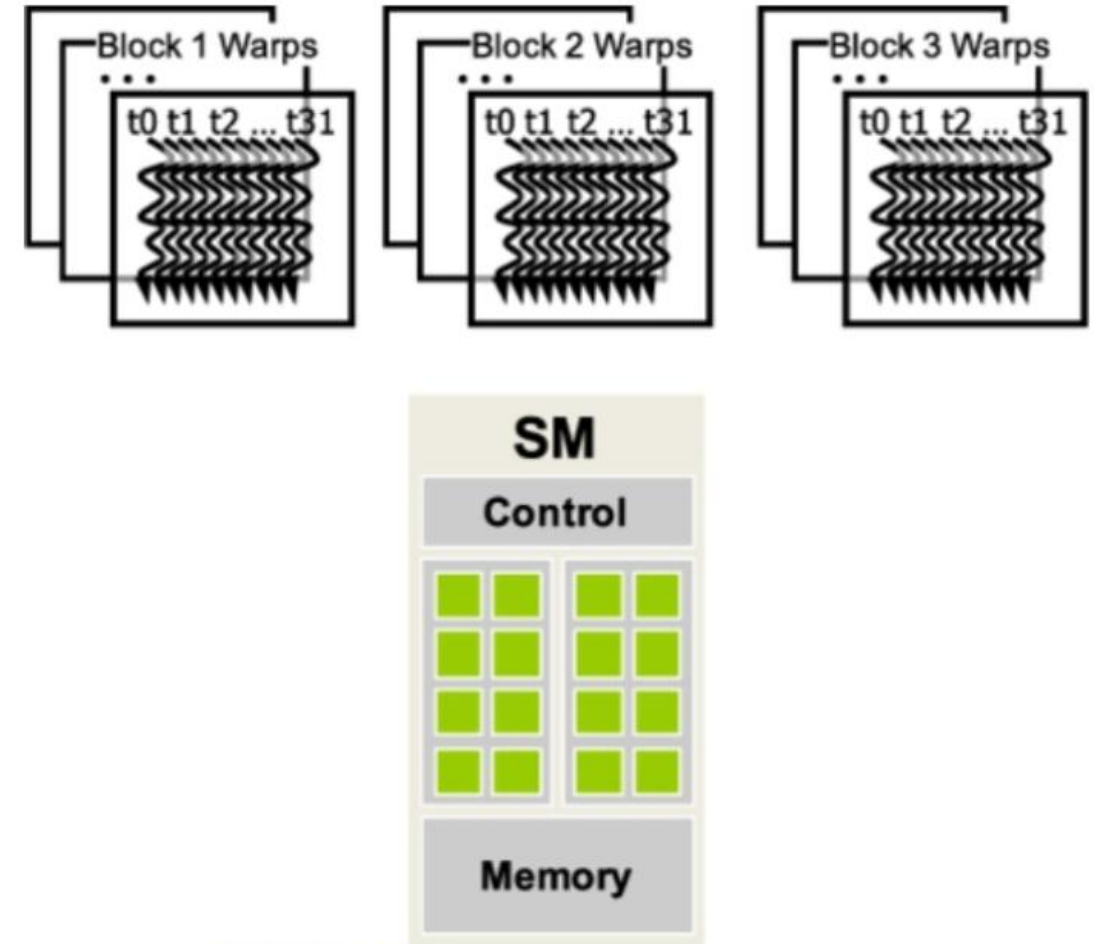


FIGURE 4.6 Blocks are partitioned into warps for thread scheduling.

Partitioning Blocks into Warps

- Partitioned based on thread indices
- **1D blocks:** Only `threadIdx.x` is used to partition. Warp 0 includes threads 0-31, Warp 1 includes threads 32-63; Warp N includes thread 32N to 32(N+1)-1
- If number of threads per block is not a multiple of 32, *the last warp will be padded with inactive threads*
- **2D blocks:** Use linearized row-major layout before partitioning based on `threadIdx.x` and `threadIdx.y` – All threads with `threadIdx.y=0` will be placed first before all threads with `threadIdx.y=1` etc.
- **3D blocks:** All threads with `threadIdx.z=0` will be placed first before all threads with `threadIdx.z=1` etc. Within each plane, threads with `threadIdx.y=0` will be placed before threads with `threadIdx.y=1` etc.

Courtesy: PMPP Textbook

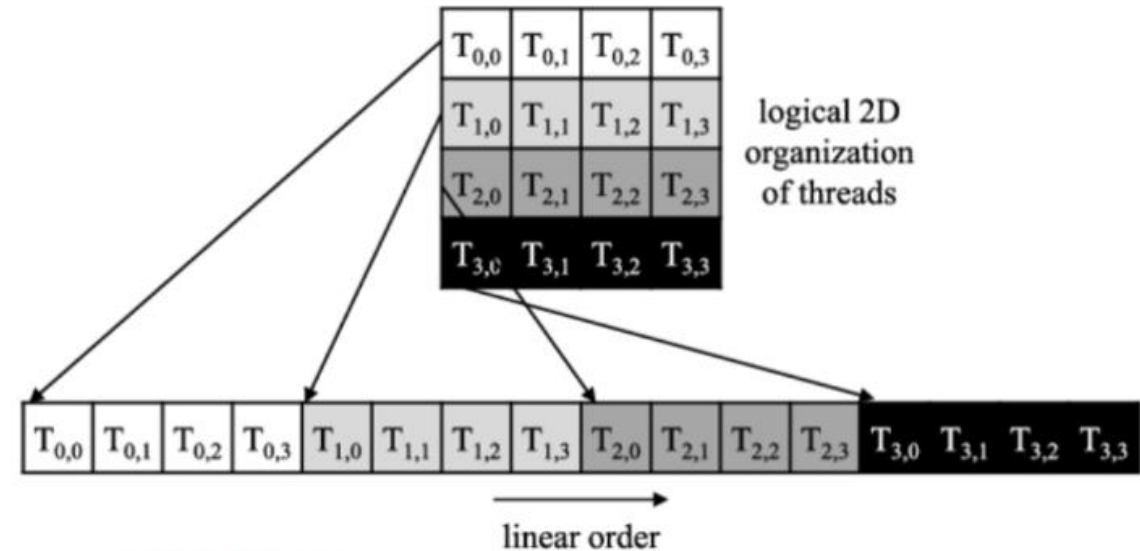
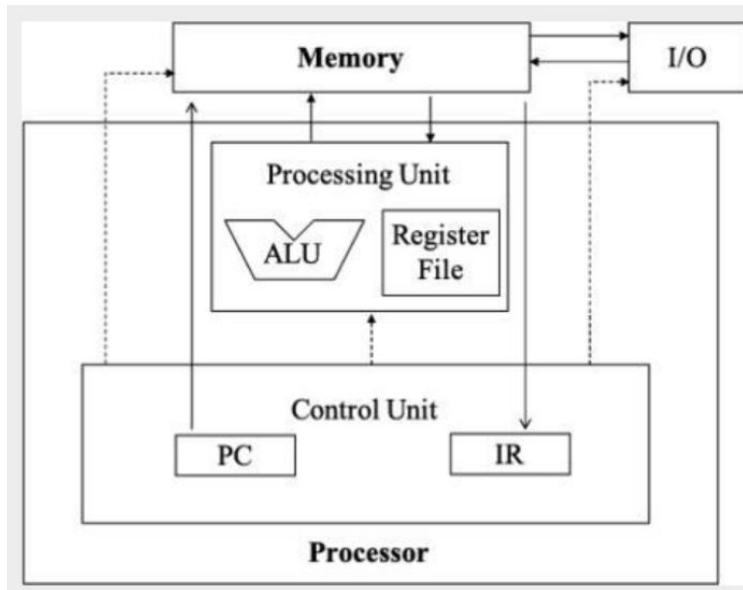


FIGURE 4.7 Placing 2D threads into linear layout.

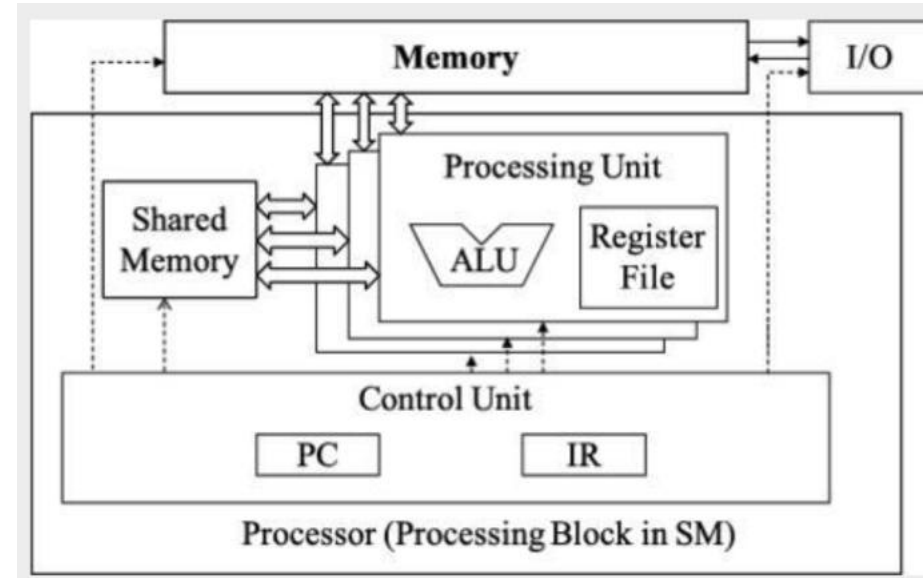
Executing Warps in CPU vs. GPU

Courtesy: PMPP Textbook

CPU



GPU



- An SM executes all warp threads using the SIMD model: At any time, one instruction is fetched and executed by all threads in the warp; called Single Instruction, Multiple Thread (SIMT).
 - Advantage: Kernel code written for scalar threads, later grouped into warps by SM to use the SIMD hardware vs. CPU where using SIMD hardware is explicit
 - Advantage: Cost of the control hardware shared across many execution units. So a larger fraction of hardware resources is dedicated to increase throughput.
- An SM is organized into multiple **processing blocks**; warps on an SM are distributed across the processing blocks. All warp threads are assigned to the same processing block
 - Example (H100): 128 SPs per SM are split into 4 processing blocks (32 SPs each)

Control Divergence

- SIMD execution works well when all threads within a warp follow the same execution path; i.e. control flow
- When threads within a warp have different execution paths (example: if-then-else based on thread index), SIMD hardware will take multiple passes: one pass for each execution path. This is called **control divergence**
 - Threads are still independent (each taking its own control path) at the cost of time needed for extra passes
- In Pascal & older architectures, passes were executed sequentially: One pass completes before another starts.
- Starting from the Volta architecture, passes may be executed concurrently (one pass could be interleaved with another. This is called **independent thread scheduling**
 - Side effect: if all threads in a warp must complete an execution phase before any moves on, a warp-level barrier synchronization `__syncwarp()` is required for correctness

Courtesy: PMPP Textbook

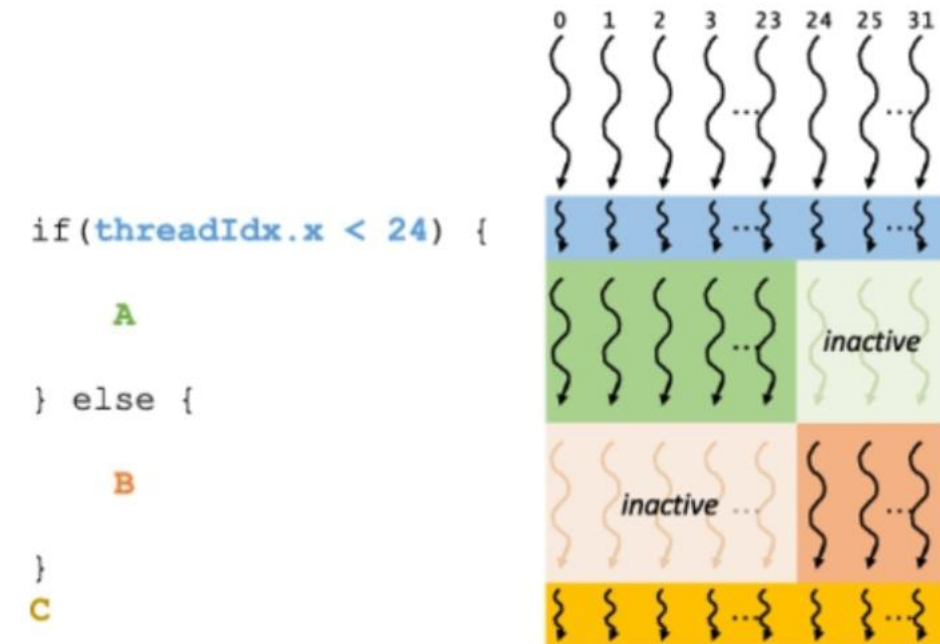


FIGURE 4.9 Example of a warp diverging at an if-else statement.

Control Divergence Examples

- Not limited to if-then-else. Could also happen in for loops.
- Sometimes needed to handle boundary conditions
- **Example:** Vector Add with vector length 1003.
 - Assume block size is 64 threads (2 warps), so we need 16 total blocks. Total threads across all blocks = $64 \times 16 = 1024$.
 - The second warp of block 15 will have the last 21 threads disabled
- Impact of boundary conditions decreases with larger vector sizes. Vector size 100: 1/4 warps will have divergence. Vector size 1000: 1/32 warps (3%) will have divergence.
- **Exercise:** Estimate warps with control divergence in the color-to-grayscale example.

Courtesy: PMPP Textbook

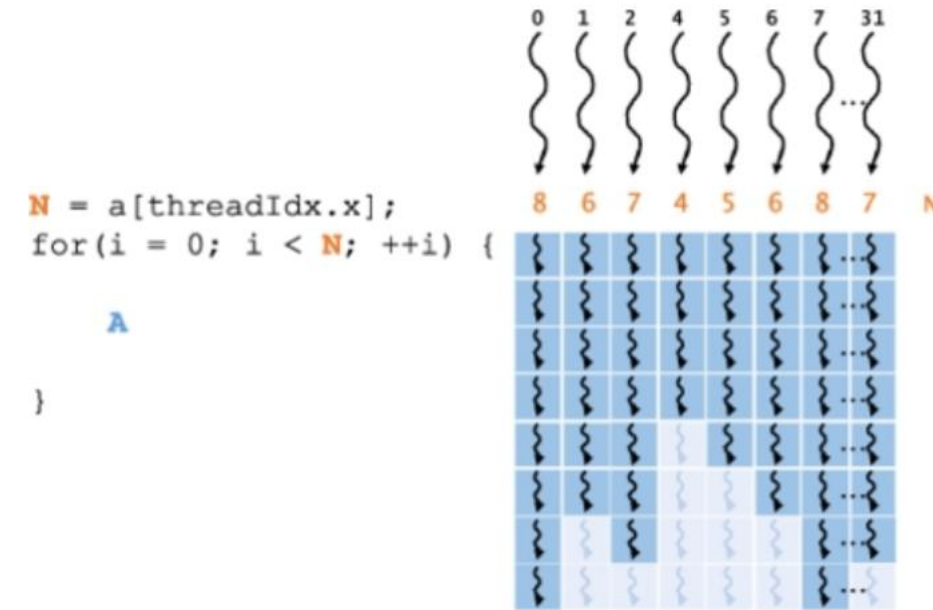


FIGURE 4.10 Example of a warp diverging at a for-loop.

Why Allocate Many Warps to Same SM?

- With 128 SPs (4 processing blocks) per SM, it can execute 4 warps simultaneously. But it can have up to 2048 threads allocated to it (16x more than SPs). Why?
- Main reason: **Latency tolerance** for Long-latency operations, e.g., global memory accesses, pipelined FP operations, conditional branches.
- When an instruction to be executed by a warp needs to wait for the result of a previous long-latency operation, the warp is not selected for execution; and another resident warp that is ready (not waiting for results of previous instruction) is selected.
 - If more than one warp is ready, priority logic selects one to execute.
- ***Fine-grained multithreading*** is the ability for each SM to select different ready warps for execution at each instant.
- With enough warps in an SM, long waiting time of warp instructions is “hidden” by executing instructions from other warps. Implication: less hardware to caches & more for FP execution
- ***Zero-overhead scheduling***: GPU's ability to put a warp that needs to wait for a long-latency instruction result to sleep and activate a warp that is ready to go without introducing any extra idle cycles in the processing units. This is done by holding all warps' execution states in hardware registers; so no need for save/restore on context switch.

Resource Partitioning and Occupancy

- **Occupancy:** Ratio between #warps assigned to an SM and the maximum number it supports
- Dynamic resource partitioning may prevent from reaching 100% occupancy.
- **SM Resources:** SPs, registers, shared memory (next week's topic), thread block slots, thread slots
- **H100 SM resources:** 32 blocks, 64 warps (2048 threads), 1024 threads/block, 64K (65,536) registers
- **RTX2000 SM resources:** 24 blocks, 48 warps (1536 threads), 1024 threads/block, 64K registers
- Examples (H100):
 - **Block size 1024 to 64** result in 2-32 blocks per SM, using all available 2048 threads (100% occupancy)
 - **Block size 32:** Need 64 blocks to fill all 2048 threads, but limited to 32 blocks/SM. Only 1024 threads will be used (50% occupancy)
 - **Block size 768:** SM can only fit 2 blocks with a total of 1536 threads (75% occupancy)
 - **Kernel uses 32 registers / thread:** 2048 threads need $32 \times 2048 = 64\text{K}$ registers, so 100% occupancy is possible
 - **Kernel uses 64 registers / thread:** 64K registers can only satisfy $64\text{K}/64 = 1024$ threads (50% occupancy)
 - **Kernel uses 33 registers / thread and block size 512:** Cannot use all 2048 threads due to register limits, so only 3 blocks are allocated per SM instead of 4 (75% occupancy)

Querying Device Resources

Courtesy: PMPP Textbook

- Allows for program flexibility to run on GPUs with varying resources

- Number of CUDA-capable devices in system:

```
1 int devCount;
2 cudaGetDeviceCount(&devCount);
```

- Properties of each CUDA device:

```
1 cudaDeviceProp devProp;
2 for(unsigned int i = 0; i < devCount; i++) {
3     cudaGetDeviceProperties(&devProp, i);
4     // Decide if device has sufficient resources/capabilities
5 }
```

- Max threads/block, threads/warp: `devProp.maxThreadsPerBlock` , `devProp.warpSize`

- Number of SMs per device: `devProp.multiProcessorCount`

- Clock Frequency: `devProp.clockRate`

- Max threads/ block dimension for x, y and z dimensions: `devProp.maxThreadsDim[0]`
`devProp.maxThreadsDim[1]` `devProp.maxThreadsDim[2]`

- Max blocks / grid dimension: `devProp.maxGridSize[0]` `devProp.maxGridSize[1]`
`devProp.maxGridSize[2]`

- Registers / SM: `devProp.regsPerBlock`

Exercises (from Textbook)

Courtesy: PMPP Textbook

- a. What is the number of warps per block?
- b. What is the number of warps in the grid?
- c. For the statement on line 04: i. How many warps in the grid are active? ii. How many warps in the grid are divergent? iii. What is the SIMD efficiency (in %) of warp 0 of block 0? iv. What is the SIMD efficiency (in %) of warp 1 of block 0? v. What is the SIMD efficiency (in %) of warp 3 of block 0?
- d. For the statement on line 07: i. How many warps in the grid are active? ii. How many warps in the grid are divergent? iii. What is the SIMD efficiency (in %) of warp 0 of block 0?
- e. For the loop on line 09: i. How many iterations have no divergence? ii. How many iterations have divergence?

```
1  __global__ void foo_kernel(int* a, int* b) {
2      unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
3      if(threadIdx.x < 40 || threadIdx.x >= 104) {
4          b[i] = a[i] + 1;
5      }
6      if(i%2 == 0) {
7          a[i] = b[i]*2;
8      }
9      for(unsigned int j = 0; j < 5 - (i%3); ++j) {
10         b[i] += j;
11     }
12 }
13 void foo(int* a_d, int* b_d) {
14     unsigned int N = 1024;
15     foo_kernel <<< (N + 128 - 1)/128, 128 >>>(a_d, b_d);
16 }
```

More Exercises (From Textbook)

Courtesy: PMPP Textbook

- Consider a hypothetical block with 8 threads executing a section of code before reaching a barrier. The threads require the following amount of time (in micro seconds) to execute the sections: 2.0, 2.3, 3.0, 2.8, 2.4, 1.9, 2.6, 2.9 and spend the rest of their time waiting for the barrier. What percentage of the threads' total execution time is spent waiting for the barrier?
- If a CUDA device's SM (streaming multiprocessor) can take up to 1536 threads and up to 4 thread blocks. Which of the following block configuration would result in the most number of threads in the SM?
a. 128 threads per block b. 256 threads per block c. 512 threads per block
d. 1024 threads per block
- Assume a device that allows up to 64 blocks per SM and 2048 threads per SM. Indicate which of the following assignments per SM is possible. In the cases where it is possible, indicate the occupancy level.
a. 8 blocks with 128 threads each b. 16 blocks with 64 threads each c. 32 blocks with 32 threads each
d. 64 blocks with 32 threads each e. 32 blocks with 64 threads each
- Consider a GPU with the following hardware limits: 2048 threads/SM, 32 blocks/SM, and 64 K (65,536) registers/SM. For each of the following kernel characteristics, specify if the kernel can achieve full occupancy. If not, specify the limiting factor.
a. The kernel uses 128 threads/block and 30 registers/thread.
b. The kernel uses 32 threads/block and 29 registers/thread.
c. The kernel uses 256 threads/block and 34 registers/thread.

Class Announcements

- **HW2: Due Tuesday Sep 22**
- **This week's reading:**
 - PMPP Chapters 3-4
 - (Skim) Nvidia Whitepaper "NVIDIA ADA LOVELACE PROFESSIONAL GPU ARCHITECTURE" https://images.nvidia.com/aem-dam/en-zz/Solutions/technologies/NVIDIA-ADA-GPU-PROVIZ-Architecture-Whitepaper_1.1.pdf
 - (Skim) Nvidia Whitepaper "NVIDIA RTX BLACKWELL GPU ARCHITECTURE" <https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf>
 - (Try out) NVIDIA, Nsight compute: occupancy calculator, <https://docs.nvidia.com/nsight-compute/NsightCompute/index.html#occupancy-calculator>
- **Next week's reading:**
 - PMPP Chapters 5-6